

14.5 A 28nm 192.3TFLOPS/W Accurate/Approximate Dual-Mode-Transpose Digital 6T-SRAM CIM Macro for Floating-Point Edge Training and Inference

Yiyang Yuan^{1,2}, Bingxin Zhang^{1,2}, Yiming Yang³, Yishan Luo^{1,2}, Qirui Chen³, Shidong Lv³, Hao Wu^{1,2}, Cailian Ma^{1,2}, Ming Li^{1,2}, Jinshan Yue¹, Xinghua Wang³, Guozhong Xing¹, Pui-In Mak⁴, Xiaoran Li³, Feng Zhang¹

¹Institute of Microelectronics of the Chinese Academy of Sciences, Beijing, China

²University of Chinese Academy of Sciences, Beijing, China

³Beijing Institute of Technology, Beijing, China, ⁴University of Macau, Macau, China

SRAM CIM macros have been developed to enhance energy efficiency (EF) in edge-AI applications. However, most research has predominantly focused on inference [1-6], with relatively little exploration into the training process. Fine-tuning, during training, is critical for improving the accuracy of neural network models, which directly impacts the user experience. Unlike remote- or cloud-based training, on-device training offers advantages such as real-time response, power saving, and user-privacy protection. The training process primarily involves two phases: feed-forward (FF) and backpropagation (BP). The FF phase resembles the inference process, while BP phase requires the multiplication of the error gradient with the transposed weight matrix. Although several studies have introduced transpose CIM (T-CIM) to support both FF and BP [7-9], several challenges remain: (1) Previous work uses separate circuits for FF and BP, thereby diminishing area and energy efficiency due to lack of multiply-accumulate (MAC) circuit reuse; (2) Prior work is limited to integer (INT) formats; research indicates that INT representation can significantly reduce accuracy during training due to its lower resolution [10]. Fortunately, developments in pre-aligned FP-CIM schemes have been made [1,11,12], but these still suffer from accuracy losses due to mantissa truncation during the pre-alignment process. (3) Reliance on analog-CIM schemes leads to accuracy losses due to process, voltage, and temperature (PVT) variations, which further degrade training accuracy. Although digital CIMs (DCIM) can mitigate these issues, optimizing the tradeoffs between SRAM arrays and MAC circuits to simultaneously achieve high-memory density (MD) and area efficiency (AF) remains challenging [1]. Increasing the number of SRAM sub-bank rows and employing bit-parallel techniques can improve MD and AF [2], while approximate computing can enhance access speed and EF [3,4], as illustrated in Fig. 14.5.1.

To address these challenges, we propose a fully-digital-transpose floating-point (FP) SRAM-CIM macro with three key features: (1) A cyclic-weight-mapping 6T-SRAM array (CWM-SRAM) that enables array transpose operations thus reusing MAC circuits for FF and BP; (2) A DCIM architecture with a signed fixed-point mantissa encoding (SFME) and a vector-wise pre-alignment (VWPA) that supports FP8, BF16, INT4, and INT8 formats to meet diverse accuracy and energy efficiency requirements during training and inference; (3) An accurate/approximate dual-mode bit-parallel MAC circuit (DMBP-MAC) that is compactly integrated within the CWM-SRAM to improve MD, access speed, AF, and EF.

Figure 14.5.2 illustrates the overall architecture of the macro, consisting of a read/write control circuit, an activation rotate-and-align circuit, and two sets of CWM-SRAM and DMBP-MAC that are organized into an upper and a lower block. The read/write-control circuit provides address decoding and read/write signals to CWM-SRAM; the activation rotate-and-align circuit aligns the elements of the activation vector with those of the weight vector, performing SFME and VWPA; the CWM-SRAM is a 1-port 4b/cell 64x64 6T-SRAM designed to support different access patterns for FF and BP. The upper and lower blocks operate separately in 4b-MAC INT4 mode, while in INT8 mode each block processes 4 MSBs and 4 LSBs. In FP8 and BF16 modes the weight and activation mantissas are pre-aligned and encoded into 4 and 8b forms; thereby, reusing the INT4 and INT8 data path. DMBP-MAC consists of 64 accurate/approximate dual-mode bit-parallel multipliers (DM-MUL) and 6 levels of dual-mode adders (DM-ADD) to generate a 14b dot product. To facilitate system integration, the macro is designed to be compatible with the standard AMBA bus interface. The AHB interface handles memory accesses, while all computing-related control signals and MAC results are transmitted via APB.

Figure 14.5.3 illustrates the scheme of CWM-SRAM. The weight matrix is stored in CWM-SRAM using a cyclic scheme, where each row of the matrix shifts its elements to the right based on its row index. During the FF phase, weights are retrieved from each row. During the BP phase, the diagonal of the weight matrix is retrieved, which corresponds to the weight matrix's transposed form. This allows for both FF and BP weights to share the 256b read port connected to the MAC circuits, simplifying the SRAM access requirements to a single-port and enabling the reuse of MAC circuits. Since the original weight vectors are shifted, due to the cyclic scheme, the activation vectors also need to be right-shifted to maintain weight-vector alignment; this adjustment is performed in the activation rotator. To implement this scheme, we propose a structure that accesses every 4b cell independently. The activation of each 4b group is controlled by separate FF and BP read-enable signals. The FF read-enable signal connects 4-row cells and the BP read-enable signal connects 4-diagonal cells. To mitigate routing congestion, we implement the diagonal

connection using the 3rd-metal layer and connect the corresponding diagonal in the 4th layer; the row connection is in the 5th layer. This method circumvents the need for numerous MAC circuits and read ports as is the case for previous T-CIM works [7,8], resulting in a reduction in area and power consumption.

Figure 14.5.4 depicts the DCIM architecture supporting FP8, BF16, INT4, and INT8 formats. FP8 is proposed as a novel FP data format to achieve higher training efficiency than BF16 [13]. FP8 can be represented in two distinct forms: E5M2 and E4M3. The E5M2 format, with a 5b exponent and a 2b mantissa, offers greater dynamic range but lower precision; the E4M3 format, with a 4b exponent and a 3b mantissa, provides a smaller dynamic range but higher precision. SFME is proposed to enable variable bit-width FP MACs. The FP weight $-1^s \cdot 2^{E-Bias} (1.M)$ is changed to $2^{E_{max}-Bias} [-1^s (0.M_n)]$ after pre-alignment, the original exponent changed to a shared exponent $E_s = E_{max} - Bias$ and the hidden bit together with the mantissa is normalized to M_n . Thus, the aligned mantissa can be represented as a signed complement fixed-point number: $M_n = -1^s \cdot 0.M_n$, with the decimal point placed behind the sign bit. The FP activation is also aligned before the FP-MAC operation. To support a unified MAC for 4 and 8b aligned mantissas, we designed a quad-mode 4b multiplier that covers all 4b partial-product combinations for 8b 2's complement multiplications. To reduce accuracy loss caused by pre-alignment mantissa truncation we used VWPA to perform pre-alignment for elements in the same vector, allowing us to benefit from the closer same-vector element distribution in the smaller truncation window.

Figure 14.5.5 illustrates the architecture of the DMBP MAC: comprising of a DM MUL based on the broken-array approximate multiplier (BAM) concept and an adder tree constructed using DM ADD that employs the lower-part OR-adder (LOA) principle [14]. The DMBP MAC operates in two modes, approximate or accurate, enabling an accuracy/efficiency trade-off depending on the application. Compared to a bit-serial approach, the bit-parallel approach offers higher speed, due to fewer compute cycles, and lower energy consumption, due to a reduced input toggle rate [2]. In approximate mode, the DM MUL discards the 6 partial sums on the right-hand side, and the DM ADD replaces the lower-part addition with a bitwise OR operation; a sparsity-aware bias adder is introduced into DM MUL to cancel the BAM offset error. Due to the shortened critical delay path and fewer toggled components, the approximate mode achieves a 12% faster computing speed and a 31% power reduction compared to the accurate mode. The mean relative-error distance (NMED) is the mean absolute value of error distance (the difference between the actual output and an ideal output) normalized to the maximum output value. To measure the effect of approximation on the computing result, 50,000 random input pairs are sampled, yielding a 5.3% NMED. The error distances exhibit a Gaussian distribution, with an average value close to zero. Additionally, a centrosymmetric layout is proposed to integrate the DMBP MAC with the CWM SRAM, optimizing the length-width ratio of macro and alleviating routing congestion.

As shown in Fig. 14.5.6, the 28nm-process FP-transpose DCIM macro achieves a measured access time of 2.5ns at 0.9V with a FP8 IN, FP8 W and BF16 OUT. The maximum FP EF is 192.3TFLOPS/W for FP8, measured at 0.55V under a 50% weight sparsity and a 50% input toggle rate in approximate mode. Compared to previous T-CIM work [7,8], this work shows significant improvements in EF and AF, attributed to the reuse of CWM-SRAM MAC circuits for FF and BP. By employing bit-parallel and approximate arithmetic in DMBP-MAC, this work exhibits shorter access times and higher EF and AF than previous non-transpose DCIM work [1,5]. With the proposed VWPA, in accurate mode, this work shows lower accuracy loss compared with layer-wise pre-align FP CIM [1]. Using SFME, this work presents the first FP CIM capable of unifying four different INT/FP formats: INT4, INT8, FP8, and BF16. Measurements using ResNet-18@CIFAR10 and VGG-19@CIFAR100, FP8 show slightly lower inference accuracy compared to INT8 and BF16, but accuracy results significantly surpass INT4. In terms of EF, FP8 notably outperforms both INT8 and BF16, making it an optimal choice for balancing EF and accuracy. Figure 14.5.7 presents the die photo and summary table.

Acknowledgement:

This work was supported in part by National Key Research and Development Program of China 2023YFB4402400; in part by the National Natural Science Foundation of China under grant U2341218; in part by the Strategic Priority Research Program of the Chinese Academy of Sciences China under grants XDB44000000 and XDA330100; The corresponding author are Xiaoran Li (xiaoran.li@bit.edu.cn) and Feng Zhang (zhangfeng_ime@ime.ac.cn).

Background: Training demand in edge devices Challenge1: Redundant MAC circuits in T-CIM

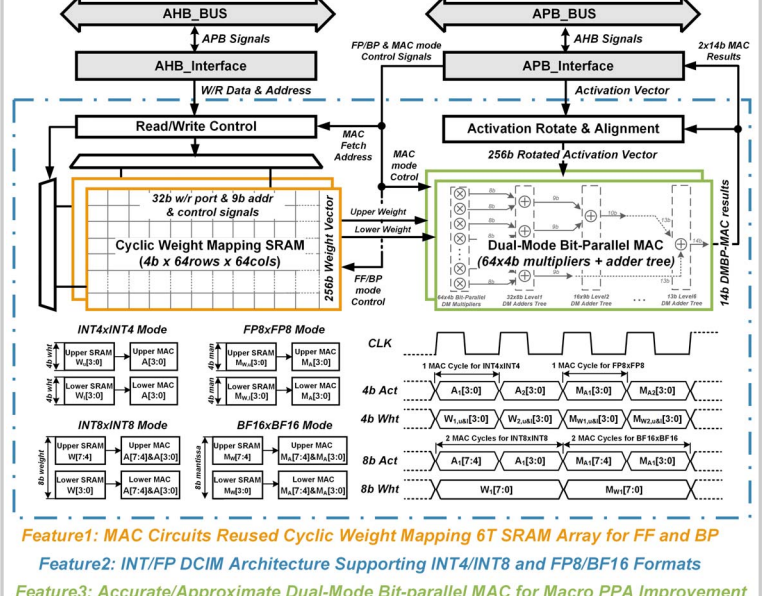
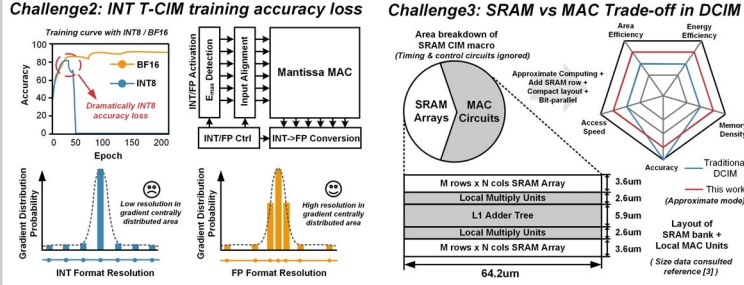
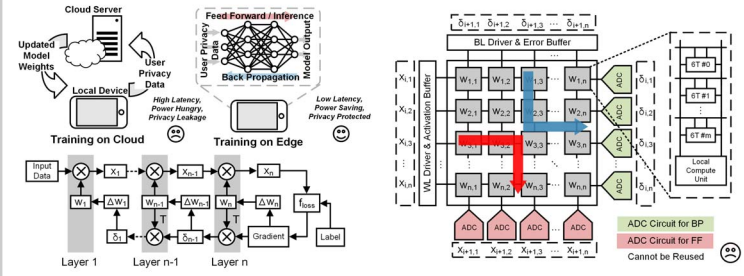


Figure 14.5.1: Challenges of transpose-CIM macro and motivation of this work.

Figure 14.5.2: Overall architecture of the proposed transpose digital-SRAM CIM.

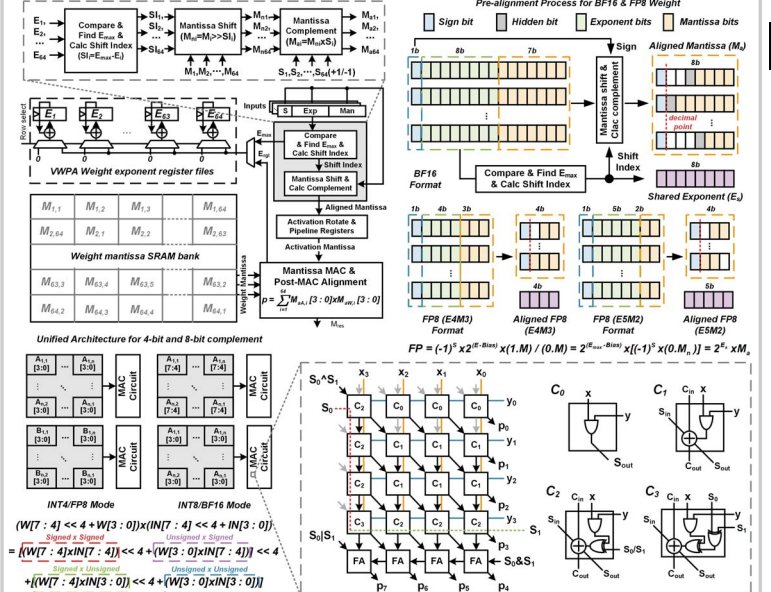
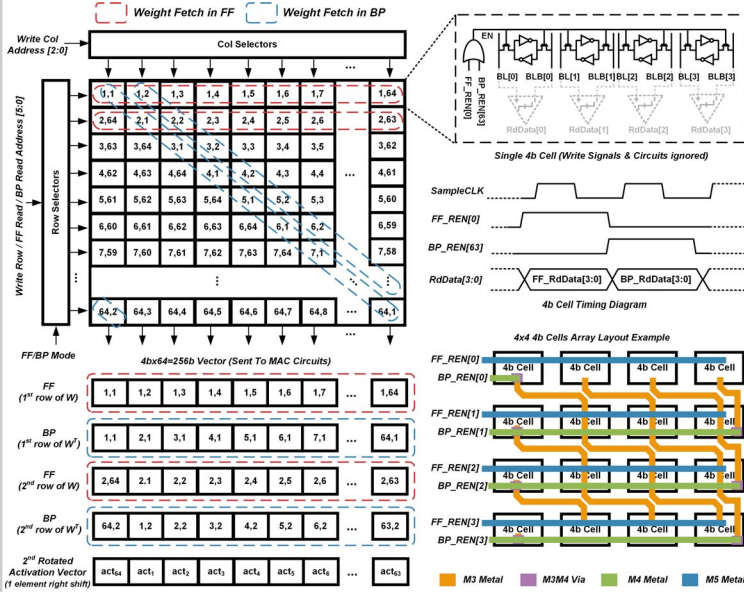


Figure 14.5.3: CWM-SRAM scheme with 6T bit cells for MAC circuit reuse.

Figure 14.5.4: SFME scheme with quad-mode 4b multiplier and VWPA.

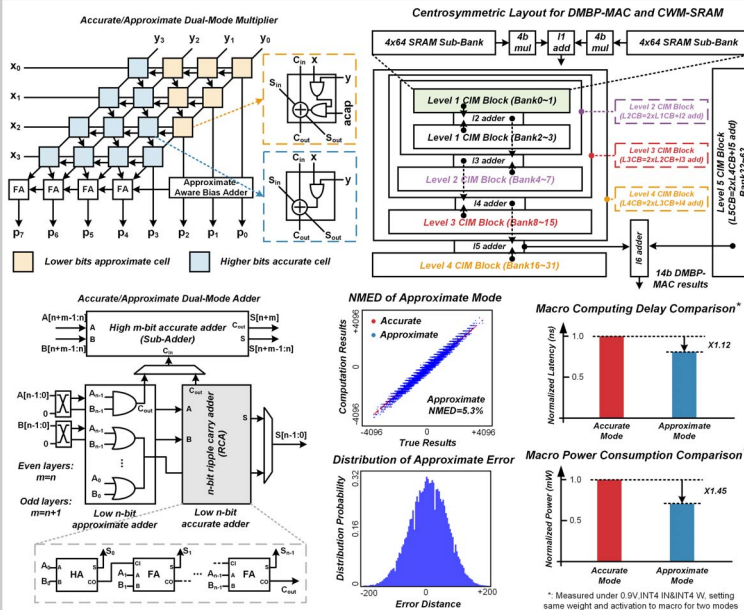


Figure 14.5.5: DMBP-MAC scheme and centrosymmetric layout.

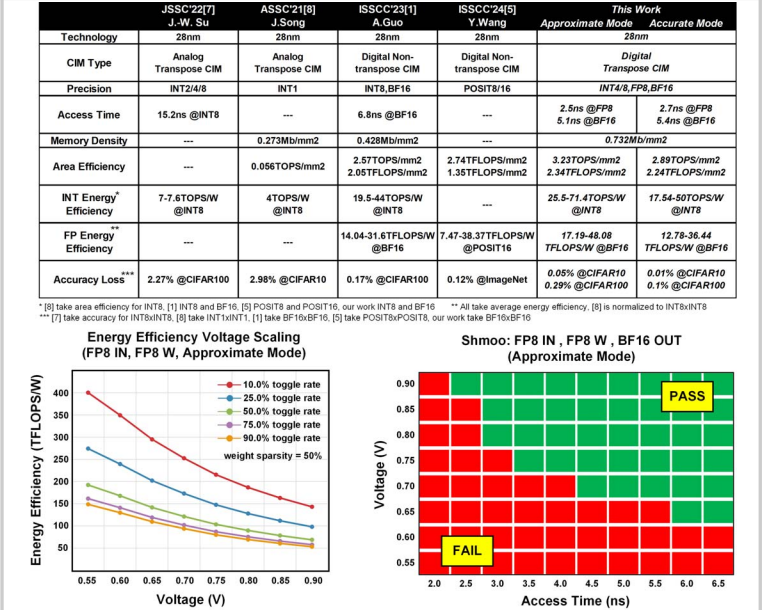


Figure 14.5.6: Comparison table, EF voltage scaling and Shmoov plot.

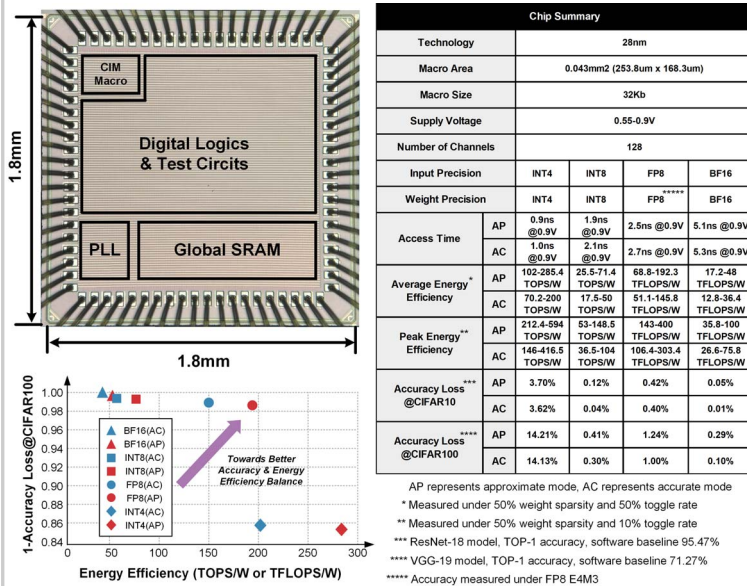


Figure 14.5.7: Die photo and summary table.

References:

- [1] A. Guo et al., "A 28nm 64-Kb 31.6-TFLOPS/W Digital-Domain Floating-Point-Computing-Unit and Double-Bit 6T-SRAM Computing-in-Memory Macro for Floating-Point CNNs," *ISSCC*, pp. 128-130, 2023.
- [2] H. Fujiwara et al., "A 3nm, 32.5TOPS/W, 55.0TOPS/mm² and 3.78Mb/mm² Fully-Digital Compute-in-Memory Macro Supporting INT12xINT12 with a Parallel-MAC Architecture and Foundry 6T-SRAM Bit Cell," *ISSCC*, pp. 572-574, 2024.
- [3] Y. He et al., "A 28nm 38-to-102-TOPS/W 8b Multiply-Less Approximate Digital SRAM Compute-In-Memory Macro for Neural-Network Inference," *ISSCC*, pp. 130-132, 2023.
- [4] D. Wang et al., "DIMC: 2219TOPS/W 2569F2/b Digital In-Memory Computing Macro in 28nm Based on Approximate Arithmetic Hardware," *ISSCC*, pp. 266-268, 2022.
- [5] Y. Wang et al., "A 28nm 83.23TFLOPS/W POSIT-Based Compute-in-Memory Macro for High-Accuracy AI Applications," *ISSCC*, pp. 566-568, 2024.
- [6] W.-S. Khwa et al., "A 16nm 96Kb Integer/Floating-Point Dual-Mode-Gain-Cell-Computing-in-Memory Macro Achieving 73.3-163.3TOPS/W and 33.2-91.2TFLOPS/W for AI-Edge Devices," *ISSCC*, pp. 568-570, 2024.
- [7] J.-W. Su et al., "Two-Way Transpose Multibit 6T SRAM Computing-in-Memory Macro for Inference-Training AI Edge Chips," *IEEE JSSC*, vol. 57, no. 2, pp. 609-624, Feb. 2022.
- [8] J. Song et al., "A 16Kb Transpose 6T SRAM In-Memory-Computing Macro based on Robust Charge-Domain Computing," *A-SSCC*, pp. 1-3, Nov. 2021.
- [9] H. Jiang et al., "CIMAT: A Compute-In-Memory Architecture for On-chip Training Based on Transpose SRAM Arrays," *IEEE Tran. on Comp.*, vol. 69, no. 7, pp. 944-954, Jul. 2020.
- [10] S. Wang et al., "Gradient Distribution-Aware INT8 Training for Neural Networks," *Neurocomputing*, vol. 541, Article 126269, Jul. 2023.
- [11] F. Tu et al., "A 28nm 29.2TFLOPS/W BF16 and 36.5TOPS/W INT8 Reconfigurable Digital CIM Processor with Unified FP/INT Pipeline and Bitwise In-Memory Booth Multiplication for Cloud Deep Learning Acceleration," *ISSCC*, pp. 255-257, 2022.
- [12] T.-H. Wen et al., "A 22nm 16Mb Floating-Point ReRAM Compute-in-Memory Macro with 31.2TFLOPS/W for AI Edge Devices," *ISSCC*, pp. 580-582, 2024.
- [13] P. Mickevicius et al., "FP8 Formats for Deep Learning," *arXiv:2209.05433*.
- [14] H. R. Mahdiani et al., "Bio-Inspired Imprecise Computational Blocks for Efficient VLSI Implementation of Soft-Computing Applications," *IEEE Tran. Cir. & Sys. I*, vol. 57, no. 4, pp. 850-862, Apr. 2010.